

AI FAQ ASSISTANT AI-Powered FAQ and Customer Support Automation

Team ID : SWTID-2026-2790

Team Leader : Neethiselvan S

Team member : Prakash raj A

Team member : SURIYA PRAKASH

Team member : SANJAY K

1. PROJECT DESCRIPTION

The AI FAQ Assistant is a secure and intelligent REST API-based application designed to simplify FAQ creation, management, searching, and customer-support automation.

The system is developed using Node.js and Express.js as the backend framework and MongoDB with Mongoose ODM for database management. JWT authentication is used for secure user sessions and bcrypt is used for password hashing.

Google Gemini AI is integrated to generate FAQ questions, answers, and topic suggestions automatically. The application provides CRUD operations, category management, FAQ search, authentication, authorization, schema validation, and centralized error handling.

2. SCENARIO-BASED CASE STUDY

2.1 Background

Customer-support teams receive a large number of repetitive questions. Manually preparing answers, organizing questions into categories, maintaining FAQ documents, and searching for existing answers can consume considerable time.

2.2 Problem Statement

- Difficulty in creating and maintaining updated FAQ documentation.
- Time-consuming manual preparation of customer-support answers.
- Difficulty organizing FAQs into suitable categories.
- Limited visibility into search keywords and matching FAQ records.
- Lack of centralized permission management.
- Difficulty protecting private FAQ information from unauthorized modification.

2.3 Proposed Solution

The proposed system provides a secure REST API that manages users, FAQs, categories, searching, and AI-generated content. Google Gemini AI automatically generates FAQ questions and answers based on a supplied topic.

2.4 System Usage

- Authenticated creators can register, log in, generate AI assistance, create FAQs, update or delete owned records, and organize FAQs by category.
- Public users can view published FAQs and search available FAQ information.

2.5 Expected Outcome

The system reduces manual FAQ-management effort by automating content generation and providing structured storage and search capabilities. Authentication protects private resources and Mongoose validation helps maintain database integrity.

3. SOFTWARE REQUIREMENTS

Requirement	Description
Operating System	Windows 10/11, macOS or Linux
Node.js	Version 18 or above
npm	Version 9 or above
Express.js	Backend REST API framework
MongoDB	NoSQL database
Mongoose	MongoDB ODM
Postman / ThunderClient	API testing
Visual Studio Code	Development environment
Google Gemini	AI-based FAQ generation

4. HARDWARE REQUIREMENTS

- Processor: Intel Core i5 8th Generation or above / AMD Ryzen 5 or equivalent.
- RAM: Minimum 8 GB; 16 GB recommended for concurrent database operations.
- Storage: Minimum 1 GB available workspace.
- Internet connection: Required for external AI service integration.

5. PROJECT ARCHITECTURE

The system follows a layered backend architecture. Incoming requests pass through the Express gateway and authentication middleware before reaching controllers, database services, or the AI integration layer.

- API Interface: API clients such as Postman or ThunderClient communicate through HTTP requests and receive JSON responses.
- Express Server Gateway: Handles HTTP methods, JSON parsing, CORS, and route mapping.
- Authentication Middleware: Extracts and validates JWT tokens and protects private routes.
- Service / Logic Layer: Manages authentication, FAQ operations, searching, and AI workflows.
- Database Interface Layer: Mongoose translates application operations into MongoDB queries.

- AI Integration Layer: Connects the application to Google Gemini for automated FAQ generation.

6. DATABASE DESIGN

The application uses MongoDB as its primary database. The major logical entities are User, FAQ, AI Generation, and Category.

6.1 User Entity

Field	Description
_id	Primary key / ObjectId
name	User name
email	Unique email address
password	Hashed password
role	User role
createdAt	Account creation time
updatedAt	Last update time

6.2 FAQ Entity

Field	Description
_id	Primary key
question	FAQ question
answer	FAQ answer
category	FAQ category
createdBy	Reference to User
createdAt	Creation timestamp
updatedAt	Update timestamp

6.3 AI Generation Entity

- topic
- generatedQuestion
- generatedAnswer
- generatedCategory
- generatedAt

6.4 Category Entity

- categoryName
- description

- totalFAQs

7. ENTITY RELATIONSHIPS

- User → FAQ: One user can create and manage multiple FAQ records.
- Category → FAQ: One category can contain multiple FAQ records.
- User → AI Generated FAQ: One user can generate multiple AI-assisted FAQs.
- FAQ → Search Results: Search operations retrieve matching FAQ records using keyword or semantic matching.

8. KEY FEATURES

- JWT Authentication and Authorization
- AI FAQ Generation using Google Gemini
- Keyword / semantic FAQ search
- Complete CRUD operations
- Category management
- Password hashing using bcrypt
- Centralized error handling
- Mongoose schema validation
- RESTful API architecture
- MongoDB integration

9. USER ROLES AND RESPONSIBILITIES

9.1 Admin

- Manage registered users.
- View, update, and delete FAQs.
- Monitor AI-generated content.
- Manage categories.
- Access complete system functionality.

9.2 Content Creator

- Create new FAQs.
- Edit personal FAQ records.
- Delete owned FAQs.
- Generate FAQs using AI.
- Organize FAQs under categories.

9.3 Authenticated User

- View public FAQs.
- Search FAQs using keywords.
- Generate AI-assisted answers.
- Access personal profile information.

9.4 Public User

- View published FAQs.
- Search FAQs.

- Cannot create, edit, or delete FAQ records.
- No access to protected API endpoints.

10. USER FLOW

User → Authentication → JWT Verification → API Request → Controller → Business Logic → MongoDB / Gemini AI → JSON Response

10.1 Registration Flow

User → Registration API → Validate Input → Hash Password → MongoDB → Response

10.2 Login Flow

User → Login API → Validate Credentials → Generate JWT → Return Token

10.3 FAQ Creation Flow

Authenticated User → JWT Verification → FAQ Request → Validation → MongoDB → FAQ Created

10.4 AI Generation Flow

Authenticated User → AI Request → Google Gemini → Generated Content → JSON Response → Optional Persistence

11. MVC ARCHITECTURE

11.1 Model Layer

The model layer defines database structures using Mongoose schemas. It manages field definitions, validation rules, defaults, and database persistence.

11.2 Controller Layer

The controller layer receives requests, validates input, coordinates models or services, performs business operations, and returns JSON responses.

11.3 View / Routing Layer

Because the application is a headless REST API, a traditional graphical View is not used. The API routing layer acts as the interface between clients and controller handlers.

12. PROJECT SETUP

12.1 Create Project Folder

```
AI-FAQ-Assistant/  
└── src/
```

12.2 Initialize npm

```
npm init -y
```

12.3 Install Dependencies

```
npm install express mongoose dotenv cors jsonwebtoken bcrypt @google/genai
```

13. PROJECT FOLDER STRUCTURE

```
AI-FAQ-Assistant/
├── src/
│   ├── server.js
│   ├── app.js
│   ├── middleware/
│   │   ├── authMiddleware.js
│   │   └── errorMiddleware.js
│   ├── models/
│   │   ├── User.js
│   │   └── FAQ.js
│   └── routes/
│       ├── authRoutes.js
│       ├── faqRoutes.js
│       └── aiRoutes.js
├── .env
├── package.json
└── README.md
```

14. BACKEND DEVELOPMENT

14.1 Server Configuration

The main server initializes environment variables, establishes the database connection, starts the Express application, and binds the server to its configured port.

14.2 Express Configuration

The Express application manages JSON parsing, CORS, API routes, middleware, and centralized error handling.

14.3 Authentication Middleware

The authentication middleware extracts the JWT from the Authorization header, verifies it, identifies the authenticated user, and permits or rejects protected requests.

14.4 Error Middleware

Centralized error middleware provides consistent responses and prevents unnecessary internal implementation details from being exposed.

15. DATABASE CONFIGURATION

MongoDB is used as the NoSQL database and Mongoose provides schema definition, validation, model creation, database queries, and document references.

The User model manages profile information and password hashing. The FAQ model stores question, answer, category, creator reference, and timestamps.

16. API ROUTES

16.1 Authentication Routes

POST /api/auth/register
POST /api/auth/login

16.2 FAQ Routes

FAQ routes support creation, retrieval, updating, deletion, and searching of FAQ records.

16.3 AI Routes

POST /api/ai/generate-faq

17. API TESTING

17.1 User Registration

POST /api/auth/register

Access: Public

```
{
  "name": "Priya Sharma",
  "email": "priya@writeflow.com",
  "password": "securepassword123"
}
```

17.2 User Login

POST /api/auth/login

Access: Public

```
{
  "email": "priya@writeflow.com",
  "password": "securepassword123"
}
```

17.3 Create FAQ

POST /api/faqs

Access: Private (Bearer JWT)

```
{
  "question": "How do I configure custom category tags?",
  "answer": "Navigate to settings panel, append keyword tags arrays, and save schema.",
  "category": "Configuration"
}
```

17.4 Search FAQ

GET /api/faqs/search?q=configure

17.5 AI FAQ Generation

POST /api/ai/generate-faq

Access: Private (Bearer JWT)

```
{
  "topic": "Mongoose schema indexing validation runtime workflow optimization"
}
```

18. PROJECT EXECUTION

- Install dependencies using npm install.
- Configure the required environment variables in .env.
- Ensure MongoDB is available.

- Start the backend development server.
- Use Postman or ThunderClient to test the API endpoints.

npm run dev

19. SECURITY IMPLEMENTATION

- JWT provides stateless authentication for protected API routes.
- bcrypt hashes passwords before storage.
- Authorization controls access to protected operations.
- Mongoose validation checks incoming data before persistence.
- Centralized error handling reduces exposure of internal error information.

20. ADVANTAGES

- Reduces repetitive FAQ creation work.
- Provides AI-assisted content generation.
- Centralizes FAQ information.
- Provides keyword-based searching.
- Protects private operations using JWT.
- Securely stores passwords using bcrypt.
- Provides structured MongoDB storage.
- Supports scalable REST API architecture.
- Maintains data consistency through schema validation.
- Separates application responsibilities through modular architecture.

21. LIMITATIONS

The supplied reference primarily documents a backend REST API. It does not establish a separate frontend user interface. It also describes semantic matching but does not provide enough implementation detail to claim a specific vector-search implementation.

22. FUTURE ENHANCEMENTS

- Advanced vector-based semantic search.
- Frontend dashboard for FAQ management.
- Analytics dashboard for frequently searched questions.
- Improved AI response customization.
- FAQ version management.
- More granular role and permission controls.
- Automated FAQ quality checking.
- Multi-language support.
- Advanced reporting and monitoring.

23. CONCLUSION

The AI FAQ Assistant provides a centralized REST API platform for FAQ management and customer-support automation. The system combines Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt, and Google

Gemini AI to provide secure authentication, structured FAQ management, AI-powered content generation, searching, and CRUD operations.

By separating authentication, business logic, database access, API routing, and AI integration into modular components, the system provides a structured foundation for developing an intelligent customer-support platform.

24. REFERENCES

- Node.js
- Express.js
- MongoDB
- Mongoose ODM
- JSON Web Token (JWT)
- bcrypt
- Google Gemini AI
- Postman / ThunderClient
- Visual Studio Code